

Tecnológico de Monterrey

José Miguel Hernández Cruz, A01804683

Ana Teresa Ordoñez Espinosa, A01798485

Miguel Ángel Buendía, A01798876

Implementación del Internet de las cosas

David Higuera Rosales

Equipo: 9

Actividad: Cierre de ETAPA 2 del RETO

5 de noviembre 2026

1. Introducción y objetivo

En esta segunda etapa de nuestro proyecto realizamos las primeras pruebas de funcionamiento del sistema portable de monitoreo biomédico, cuyo propósito es adquirir información fisiológica para facilitar el seguimiento de variables relacionadas con la salud del usuario.

Para ello, utilizamos el microcontrolador ESP32 junto con el sensor MAX30102, el cual se encarga de obtener señales ópticas de luz roja e infrarroja, y el sensor LM75BD para registrar temperatura. Este último sustituye al DS18B20 que habíamos contemplado inicialmente en la Etapa 1, debido a su disponibilidad y compatibilidad con la comunicación I2C.

Además de comprobar el funcionamiento de los sensores, realizamos pruebas de conexión WiFi y comunicación HTTP, así validamos la capacidad del ESP32 para conectarse a una red y responder a solicitudes externas.

2. Materiales utilizados

Para realizar las pruebas utilizamos los siguientes componentes:

- ESP32 NodeMCU-32S.
- Sensor MAX30102 (MH-ET LIVE).
- Sensor de temperatura LM75BD.
- Protoboard.
- Cables Jumper.
- Cable USB-C de datos.
- Computadora con Arduino IDE 2.3.10.
- Celular (en este caso se utilizó un iPhone) como punto de acceso WiFi.

El MPU6050 continúa formando parte de nuestra propuesta para el análisis de movimiento y calidad del sueño, sin embargo se integrará en pruebas posteriores.

3. Montaje físico y diagrama de conexiones

Conectamos ambos sensores al ESP32 mediante el protocolo I2C, utilizando los pines GPIO 21 para datos (SDA) y GPIO 22 para reloj (SCL). También compartimos la alimentación de 3.3 V y la conexión a tierra (GND).

Componente	Pin del sensor	Conexión ESP32
MAX30102	VIN	3V3
MAX30102	GND	GND
MAX30102	SDA	GPIO 21
MAX30102	SCL	GPIO 22
LM75BD	VCC	3V3
LM75BD	GND	GND
LM75BD	SDA	GPIO 21
LM75BD	SCL	GPIO22
LM75BD	OS	sin conectar

Ambos dispositivos comparten el mismo bus I2C, puesto que cuentan con direcciones diferentes. Gracias a esta configuración, es posible recibir sus lecturas mediante el ESP32 sin utilizar pines de comunicación independientes para cada sensor.

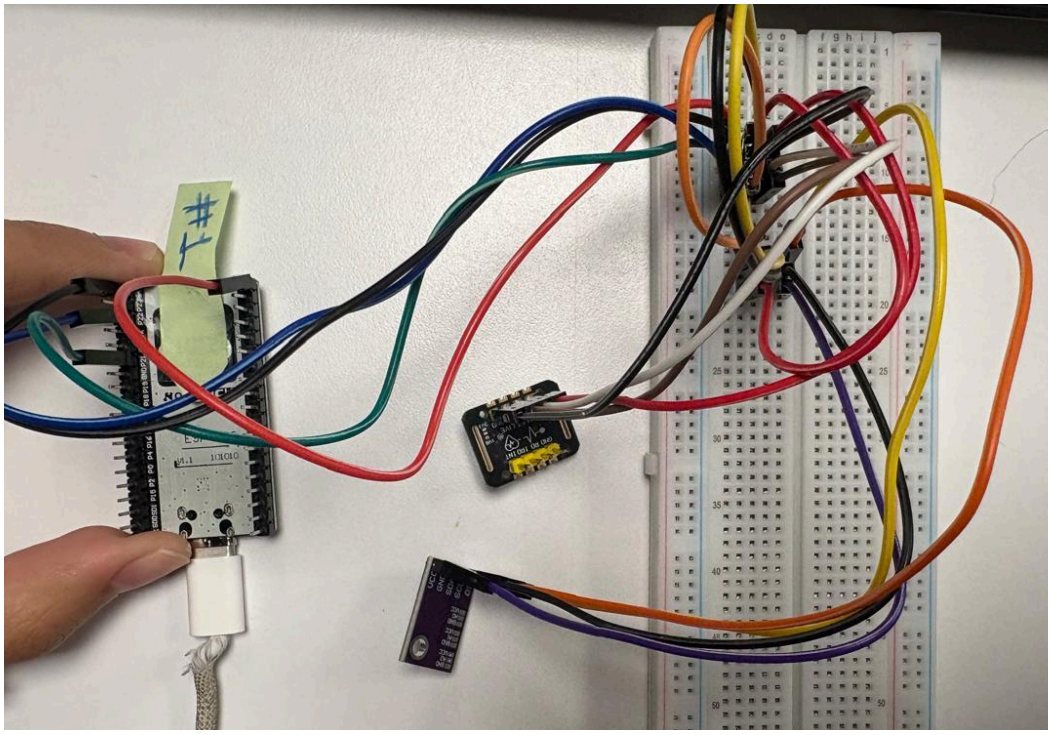


Figura 1. Montaje físico de los sensores MAX30102 y LM75BD conectados al ESP32.

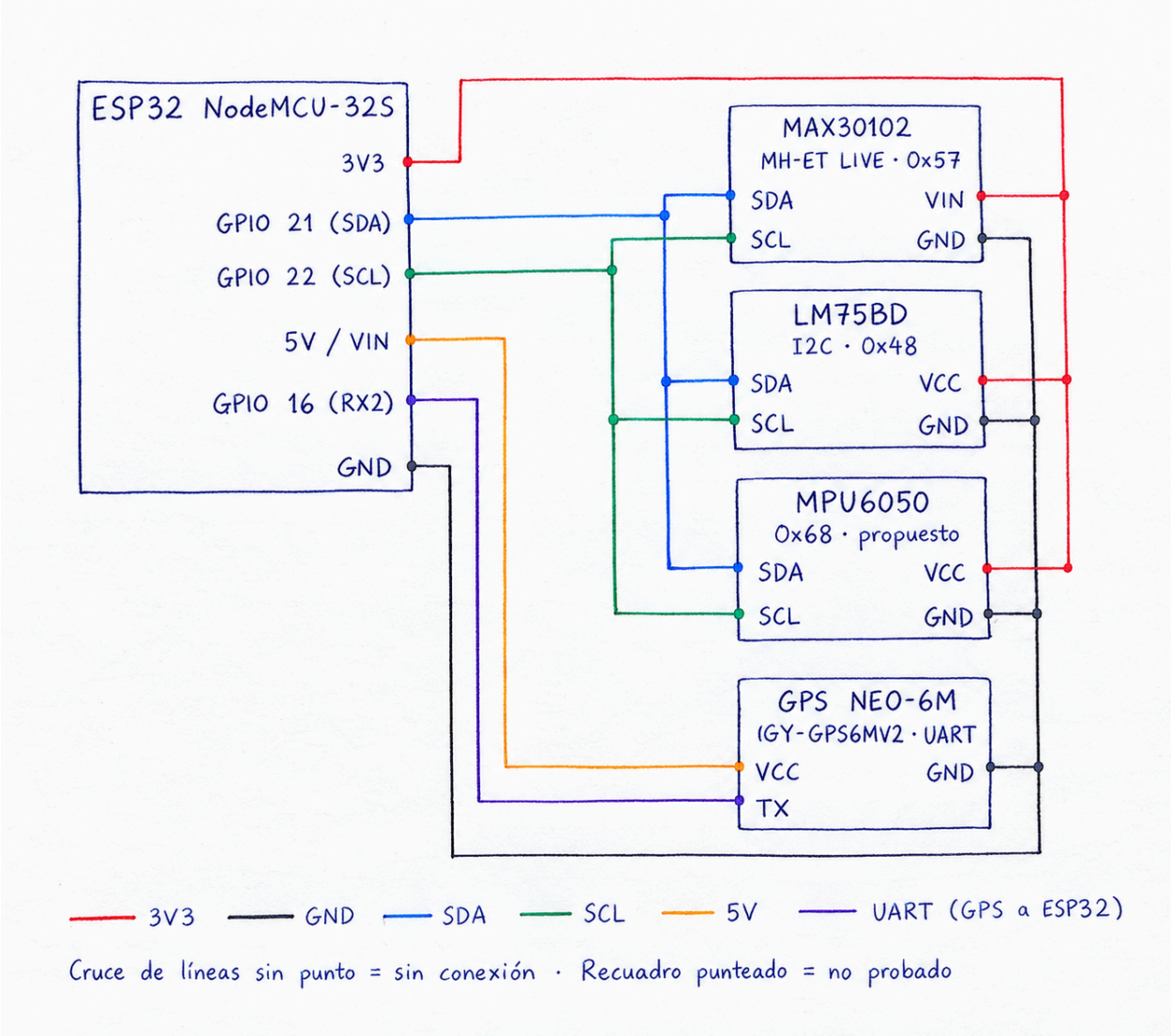


Figura 2. Borrador general del diagrama de conexiones.

4. Primera prueba: detección de sensores mediante I2C

Para comprobar el montaje y las conexiones correspondientes estuvieran correctas, llevamos a cabo un programa de escaneo I2C utilizando la biblioteca Wire.h. El cual permitió identificar los dispositivos que estaban respondiendo a las solicitudes del ESP32.

En el monitor serial obtuvimos las siguientes direcciones:

Sensor	Dirección I2C
LM75BD	0x48
MAX30102	0x57

Gracias a estos resultados fue posible confirmar que ambos sensores eran reconocidos por el ESP32 y que existía comunicación a través del bus I2C.

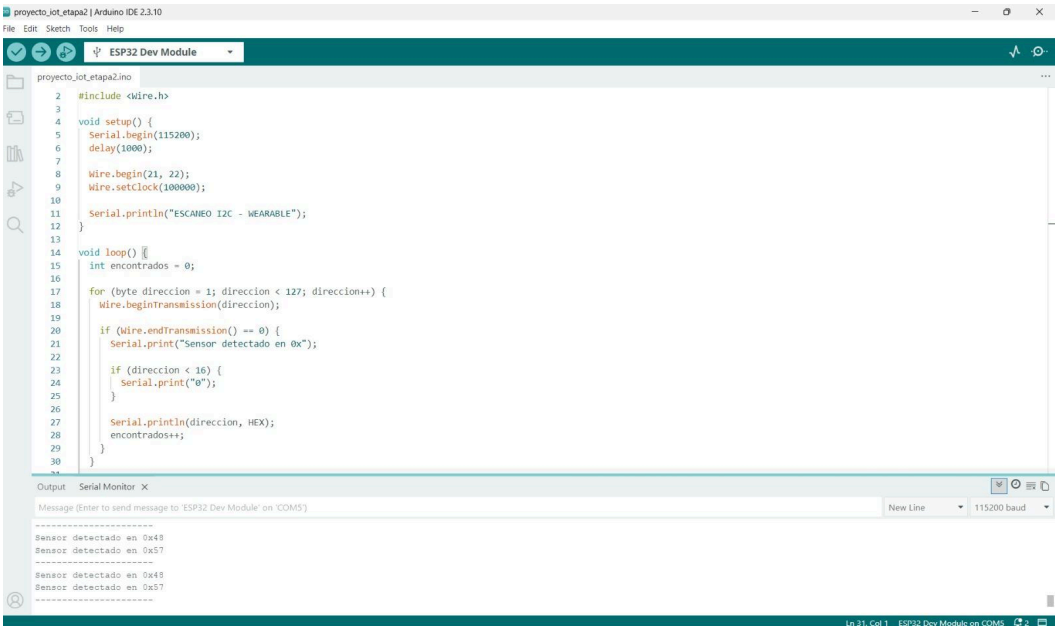


Figura 3. Detección de los sensores mediante el monitor serial de Arduino IDE.

5. Segunda prueba: lectura de IR, Red y temperatura

Después de verificar la comunicación I2C, desarrollamos un segundo programa para obtener al mismo tiempo las señales ópticas del MAX30102 y la temperatura del LM75BD.

Utilizamos la biblioteca SparkFun MAX3010x para obtener las lecturas de luz roja (Red) e infrarroja (IR), mientras que para el LM75BD hicimos uso de Wire.h y la lectura de su registro de temperatura.

Durante una de las pruebas con el dedo cerca del MAX30102 obtuvimos los siguientes resultados:

Variable	Valor registrado

Luz roja (Red)	53,580
Luz infrarroja (IR)	85,818
Temperatura	25.125 °C

Los resultados mostraron que los sensores pueden proporcionar datos simultáneamente. En el caso del MAX30102, las lecturas corresponden a intensidades ópticas digitalizadas, por lo que todavía no representan directamente la frecuencia cardíaca ni el porcentaje de saturación de oxígeno. Por otro lado, la lectura del LM75BD corresponde a la temperatura del sensor, por lo que no se debe interpretar directamente como la temperatura corporal del usuario.

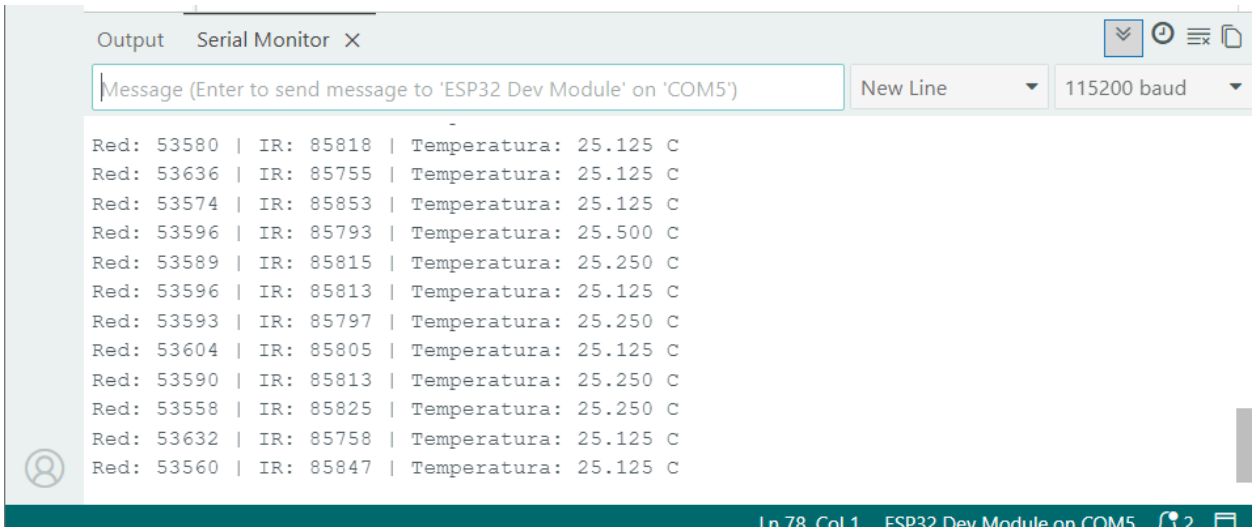


Figura 4. Lectura simultánea de Red, IR y temperatura.

6. Análisis del ruido de la señal MAX30102

Para analizar la estabilidad de las señales ópticas realizamos tres pruebas: sin colocar el dedo sobre el sensor, con el dedo apoyado y quieto, y con el dedo en movimiento.

Se registraron los siguientes resultados:

Condición	**Red**	**IR**
Sin dedo	561-688	330-677
Dedo apoyado y quieto	98,496	104,899-113,878
Dedo en movimiento	3,253-108,049	11,597-120,185

Observamos que, al mantener el dedo quieto, las señales presentaban variaciones relativamente menores. Sin embargo, al moverlo, se registraron cambios bruscos y mucho más

amplios en los valores de ambas longitudes de onda.

Esto nos permitió identificar la presencia de artefactos de movimiento que afectan la estabilidad de las señales. Aunque parte de las variaciones también puede relacionarse con la señal pulsátil y a la estabilidad del contacto con la piel, por lo cual los resultados muestran la importancia de mantener el sensor en una posición estable. Consideramos que, para mejorar las lecturas en nuestro wearable, será necesario implementar técnicas de filtrado digital y diseñar una sujeción la cual sea capaz de reducir el desplazamiento del sensor durante su uso.

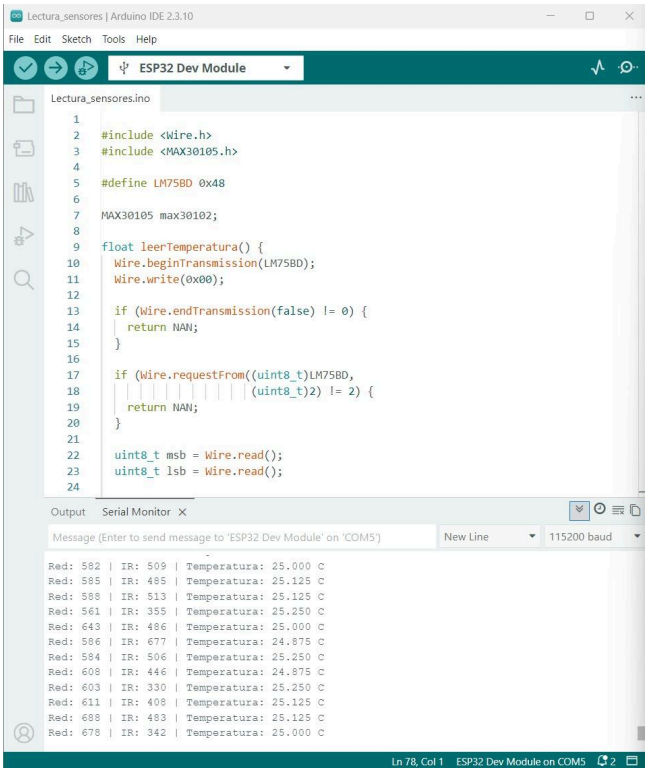


Figura 5. Lecturas del MAX30102 sin colocar el dedo.

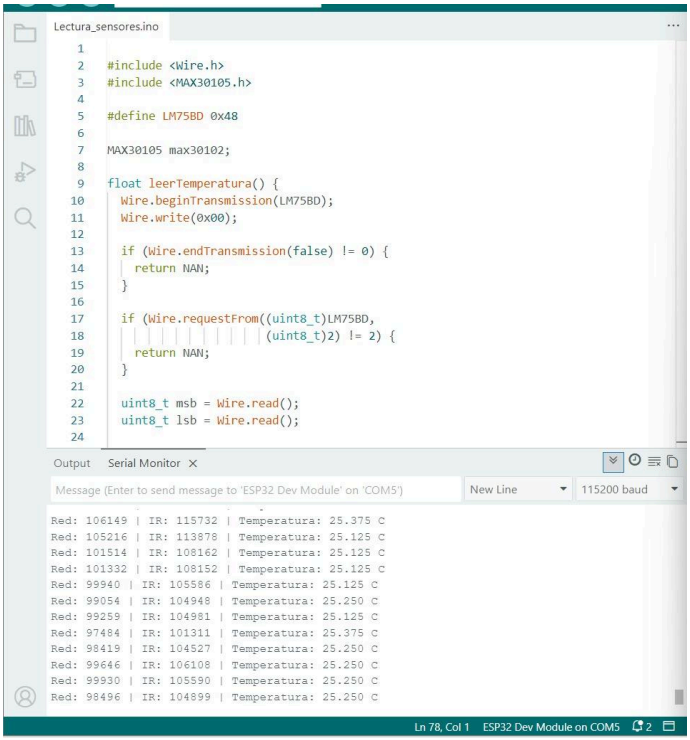


Figura 6. Lecturas con el dedo apoyado y quieto.

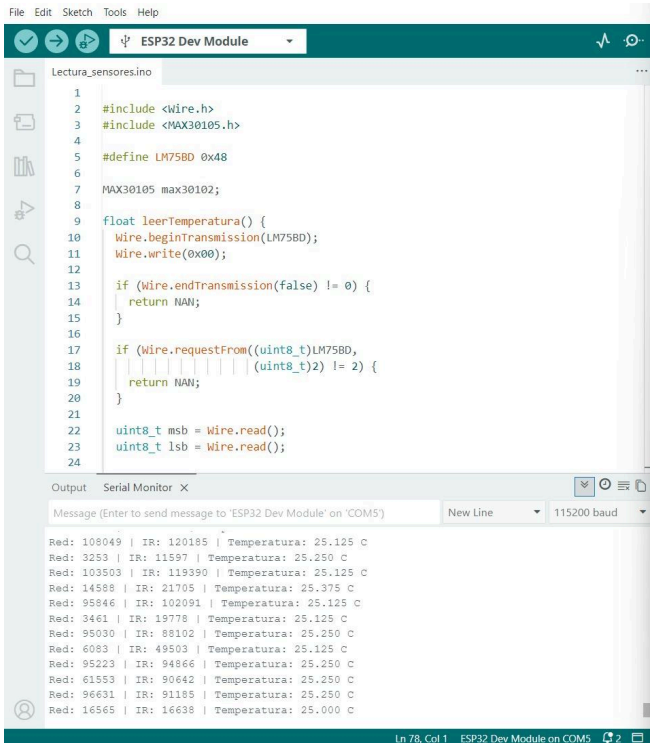


Figura 7. Lecturas con el dedo en movimiento.

7. Preguntas de reflexión

7.1. ¿Por qué el MAX30102 necesita dos longitudes de onda de luz (rojo e infrarrojo) y no le bastaría con una sola para medir SpO_2?

El MAX30102 utiliza dos longitudes de onda porque la hemoglobina oxigenada y la desoxigenada absorben la luz roja e infrarroja de manera diferente. Al analizar y comparar las variaciones de ambas señales, se puede estimar la saturación de oxígeno en la sangre.

Si utilizáramos solamente una longitud de onda, no contaríamos con suficiente información para distinguir adecuadamente el comportamiento de ambas formas de hemoglobina. Por lo tanto, la combinación de luz roja e infrarroja es de suma importancia para realizar la estimación de SpO_2.

7.2. Si tuviéramos que elegir entre el DS18B20 y el MLX90614 para un wearable de muñeca que se utiliza todo el día, ¿cuál elegiríamos y por qué?

Para un wearable que permanece colocado en la muñeca durante varias horas, elegiríamos el DS18B20 en una versión compacta que pueda tener buen contacto térmico, debido a que permite realizar mediciones de temperatura mediante contacto y resulta relativamente sencillo de integrar con un microcontrolador.

El MLX90614 tiene la ventaja de medir temperatura por radiación infrarroja sin contacto directo, pero requiere mantener condiciones adecuadas de orientación, campo de visión y estabilidad térmica. Estas condiciones pueden ser más difíciles de controlar en un dispositivo que esta en constante movimiento junto con el usuario.

Sin embargo, en nuestro prototipo decidimos utilizar el LM75BD, ya que contamos con este componente y es compatible con el protocolo I2C que también utiliza el MAX30102. Reconocemos que será necesario evaluar su comportamiento térmico y la precisión de las mediciones sobre la piel.

7.3. ¿Qué otro actuador, además de un LED o un motor de vibración, podría ser útil para notificar a una persona con discapacidad visual o auditiva?

Una alternativa sería integrar un pequeño altavoz que reproduzca mensajes de voz o alertas sonoras para usuarios con discapacidad visual. De esta manera, el sistema podría comunicar información importante sin depender de una pantalla.

Para personas con discapacidad auditiva, también podríamos considerar un mecanismo táctil activado mediante un pequeño servomotor que genere una señal que sea perceptible al tacto, o una pantalla con mensajes visuales. La elección dependería de las necesidades y preferencias de cada usuario.

8. Prueba de conectividad WiFi del ESP32

Para verificar la capacidad de comunicación inalámbrica del ESP32, desarrollamos un programa utilizando las bibliotecas WiFi.h y WebServer.h.

Durante la prueba configuramos un iPhone como punto de acceso personal y conectamos el ESP32 a esta red, utilizando una conexión WiFi compatible con la banda de 2.4 GHz. Después de cargar el programa en Arduino IDE, el monitor serial mostró el mensaje de conexión exitosa y la dirección IP asignada por la red. Esto confirmó que el ESP32 podía conectarse a una red inalámbrica y establecer comunicación IP.

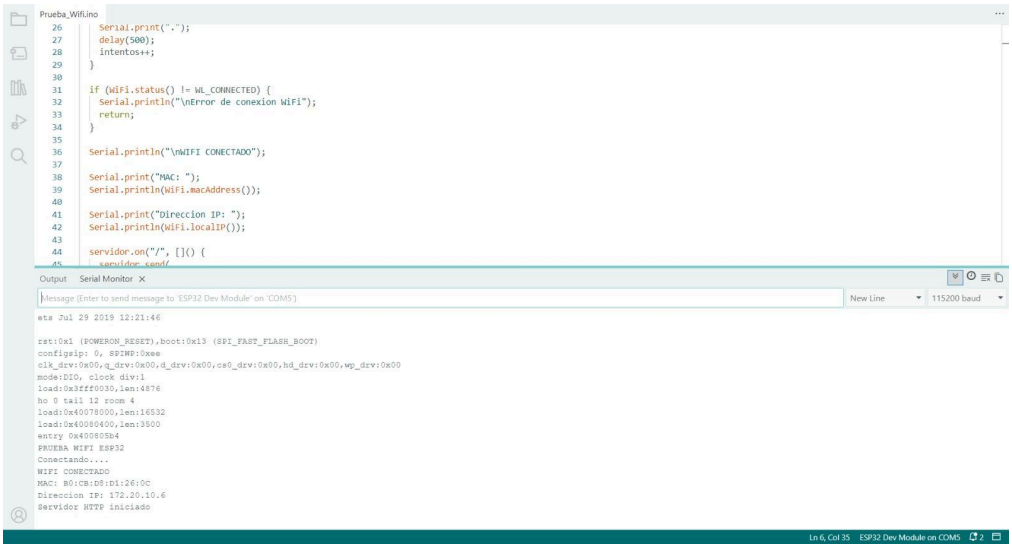


Figura 8. Conexión WiFi exitosa del ESP32.

9. Justificación de la tecnología de acceso WiFi

Seleccionamos WiFi como tecnología de comunicación para nuestro prototipo porque permite conectar el ESP32 a una red local y facilita el envío de información a aplicaciones de monitoreo.

El microcontrolador ESP32 que utilizamos es compatible con los estándares IEEE 802.11 b/g/n en la banda de 2.4 GHz. Aunque existen tecnologías más recientes, como 802.11ac, esta última no es compatible con nuestra versión del ESP32.

Para justificar nuestra elección, comparamos WiFi con otras tecnologías de comunicación:

Tecnología	**Ventajas**	**Limitaciones**
WiFi 802.11 b/g/n	Conectividad IP, buena	Mayor consumo energético y

	velocidad y fácil integración con plataformas IoT	dependencia de una red
Bluetooth Low Energy	Bajo consumo energético, adecuado para wearables	Generalmente requiere un dispositivo cercano o pasarela para acceder a internet
LoRa	Largo alcance y bajo consumo en aplicaciones de transmisión ocasional	Menor velocidad de datos y necesidad de infraestructura compatible
WiFi 802.11ac	Mayor capacidad de transmisión en equipos compatibles	No es compatible con nuestro ESP32 clásico

Después de comparar estas alternativas, consideramos que WiFi es la opción más conveniente para esta etapa del proyecto, ya que nos permite realizar pruebas de comunicación directa y preparar la integración futura con un broker MQTT y un dashboard de monitoreo.

Aunque reconocemos que Bluetooth Low Energy podría ser más conveniente para reducir el consumo energético de una versión final del wearable, actualmente nos enfocamos y priorizamos más la facilidad de conexión, las herramientas disponibles y la compatibilidad con nuestra arquitectura IoT previamente planteada.

10. Prueba de conectividad mediante HTTP 200

Para comprobar que el ESP32 podía comunicarse con otros dispositivos conectados a la misma red, implementamos un servidor HTTP mediante la biblioteca WebServer.h. Posteriormente, conectamos nuestra computadora al mismo punto de acceso WiFi y realizamos una solicitud HTTP utilizando la dirección IP asignada al ESP32. Al acceder desde el navegador obtuvimos el mensaje:

HTTP 200 OK - ESP32 conectado

Después verificamos la respuesta mediante PowerShell y confirmamos que el servidor devolvía el código de estado:

HTTP/1.1 200 OK

Este código indica que la solicitud fue recibida y ejecutada correctamente por el servidor. Con ello comprobamos la comunicación entre la computadora y el ESP32 a través de la red local.

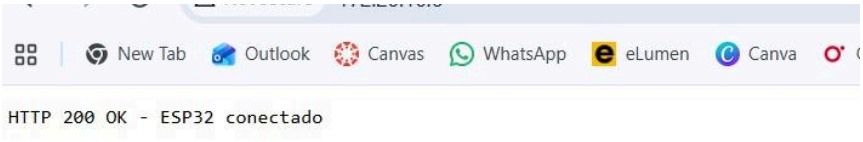


Figura 9. Respuesta del ESP32 desde el navegador web.

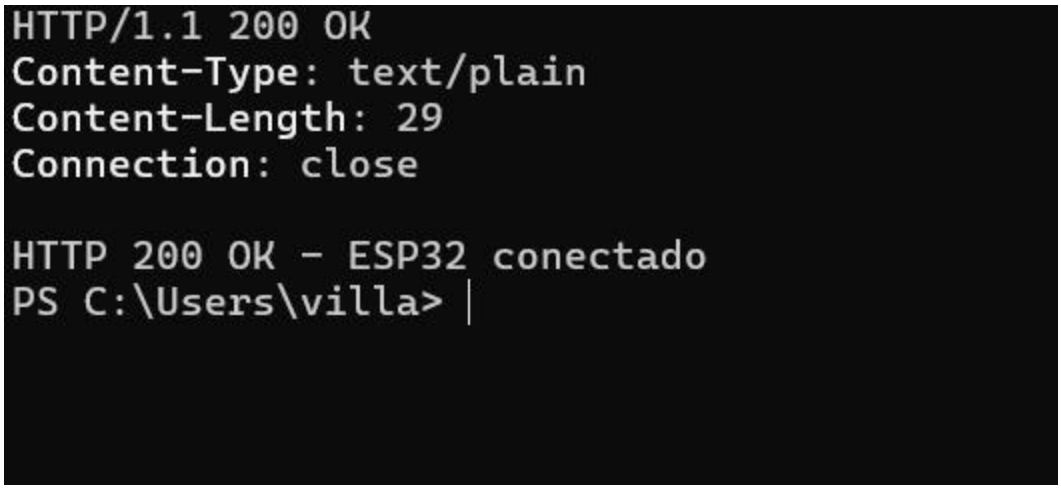


Figura 10. Verificación de la respuesta HTTP mediante PowerShell.

11. Relación con la arquitectura propuesta en la Etapa 1

Las pruebas realizadas representan un avance en la implementación de nuestra arquitectura IoT de cuatro capas. En la capa de percepción, comprobamos la adquisición datos mediante los sensores MAX30102 y LM75BD, mientras que en la capa de red validamos la conectividad WiFi del ESP32.

En las siguientes etapas continuaremos con la integración del MPU6050, el envío de información mediante MQTT, el procesamiento y almacenamiento de mediciones y el desarrollo de un dashboard para visualizar los datos. Estas tecnologías forman parte de nuestra propuesta original y las cuales aún no se implementan en esta etapa.

12. Conclusiones

Durante esta segunda etapa logramos realizar el montaje y comprobar el funcionamiento de los sensores MAX30102 y LM75BD conectados al ESP32. Las pruebas nos permitieron obtener lecturas simultáneas de luz roja, infrarroja y temperatura, además fue posible identificar las variaciones que se producen en las señales ópticas cuando existe movimiento.

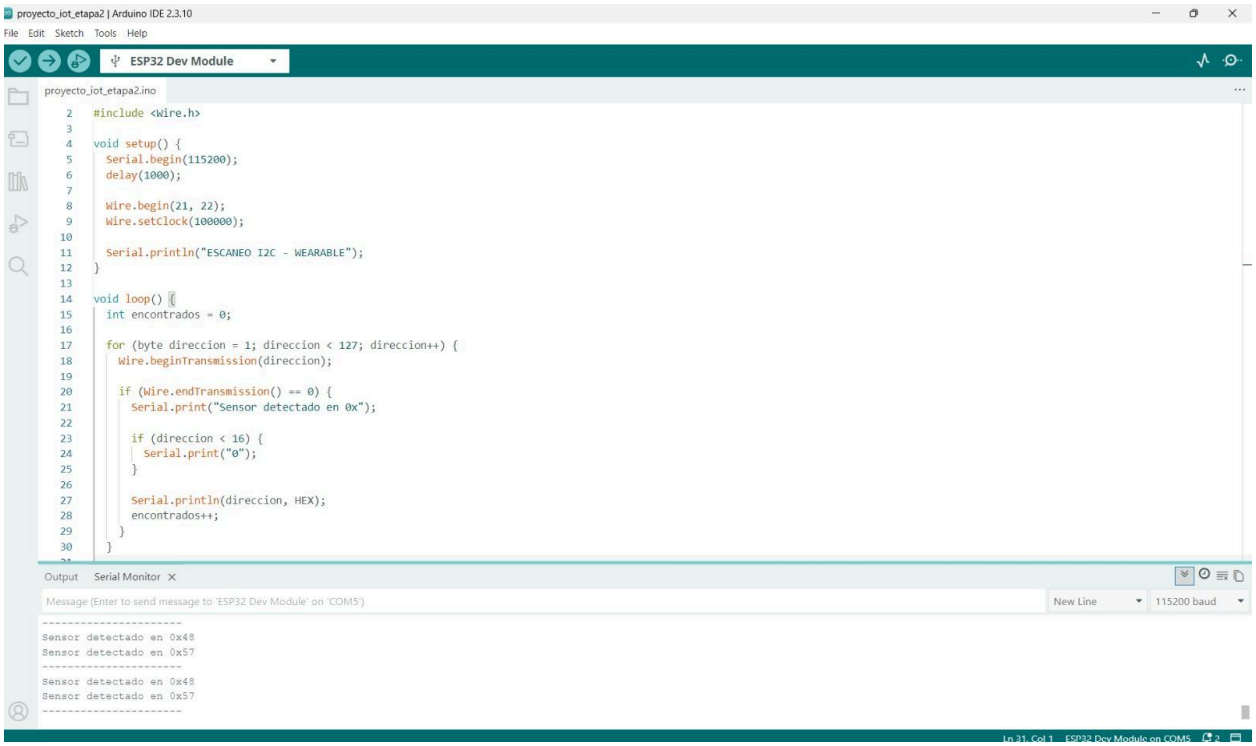
También comprobamos que el ESP32 puede conectarse correctamente a una red WiFi y responder a las solicitudes HTTP, obteniendo una respuesta exitosa con el código 200 OK. Estos resultados representan un avance importante en el desarrollo de nuestro sistema portable de monitoreo biomédico, ya que validamos las primeras funciones que nos permiten adquirir datos y conectarse adecuadamente para continuar con la integración del prototipo.

Como siguientes pasos, planeamos incorporar el MPU6050, mejorar la estabilidad de las mediciones e implementar la transmisión de datos hacia nuestra plataforma de monitoreo.

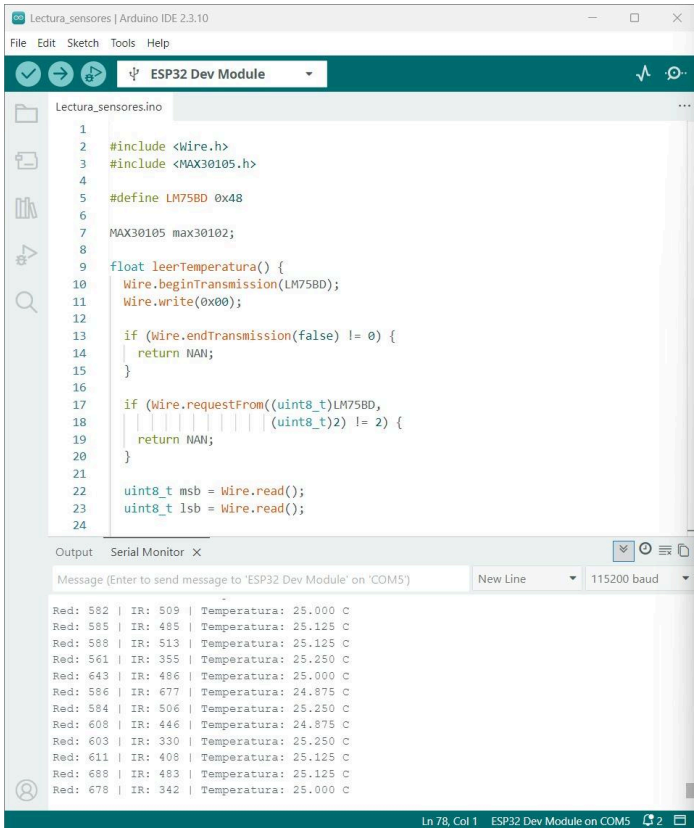
13. Anexos: códigos utilizados

Anexo A. Escaneo I2C

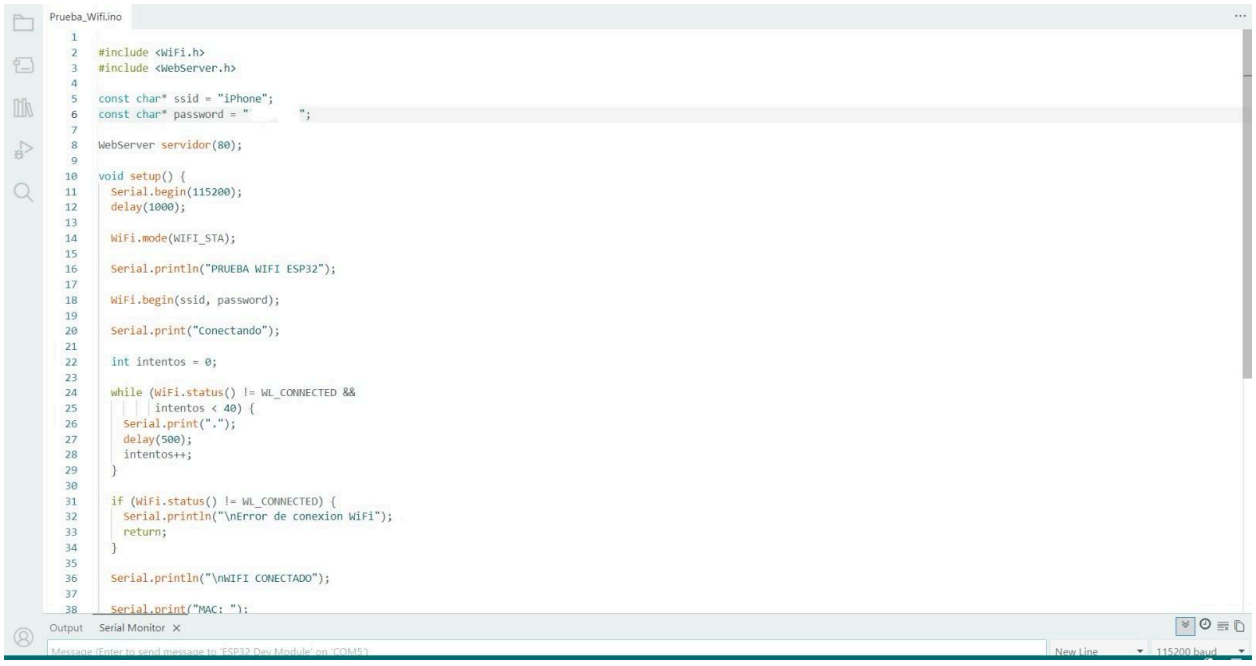




Anexo B. Lectura simultánea de sensores



Anexo C. Prueba WiFi y HTTP




Serial.print("MAC: ");
Serial.println(WiFi.macAddress());
Serial.print("Direccion IP: ");
Serial.println(WiFi.localIP());
servidor.on("/", []() {
servidor.send(
200,
"text/plain",
"HTTP 200 OK - ESP32 conectado"
);
Serial.println("Solicitud HTTP: 200 OK");
});
servidor.begin();
Serial.println("Servidor HTTP iniciado");
}
void loop() {
if (WiFi.status() == WL_CONNECTED) {
servidor.handleClient();
}
}
Output
Serial Monitor
Message (Enter to send message to 'ESP32 Dev Module' on 'COM5')
New Line
115200 baud
Ln 6, Col 35
ESP32 Dev Module on COM5
2

Referencias

Analog Devices. (s. f.). DS18B20: Programmable resolution 1-Wire digital thermometer. https://www.analog.com/en/products/ds18b20.html

Analog Devices. (s. f.). MAX30102: High-sensitivity pulse oximeter and heart-rate sensor for wearable health. https://www.analog.com/en/products/max30102.html

Espressif Systems. (s. f.). ESP32-WROOM-32 datasheet. https://documentation.espressif.com/esp32-wroom-32_datasheet_en.html

Melexis. (s. f.). MLX90614: Digital non-contact infrared thermometer. https://www.melexis.com/en/product/MLX90614/Digital-Plug-Play-Infrared-Thermometer-TO-Can

NXP Semiconductors. (2015). LM75B: Digital temperature sensor and thermal watchdog [Hoja de datos]. https://www.nxp.com/docs/en/data-sheet/LM75B.pdf

SparkFun Electronics. (s. f.). SparkFun MAX3010x sensor library [Código fuente]. GitHub. https://github.com/sparkfun/SparkFun_MAX3010x_Sensor_Library